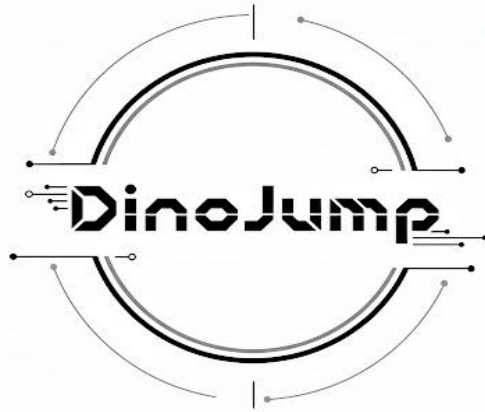




— A Python-based Arcade Game —

Project By:

Venkata Sai Kushal Reddy Pujari

(XII B)

Certificate

This is to certify that Venkata Sai Kushal Reddy Pujari, Roll No: _____ of Chrysalis High, Kadugodi, Bangalore has successfully completed the Computer Project titled "DinoJump Game" towards the partial completion of the Practical Examination of AISSCE 2025-26 as prescribed by the CBSE.

**Signature of Internal
Examiner**

**Signature of External
Examiner**

**Signature of
Principal**

Acknowledgement

Foremost, I would like to thank CBSE for organizing such practical and interesting projects which enlighten further improvement towards the subject. I extend my sincere gratitude to our respected principal, Ms. Singupilla Ratna Kumari ma'am for providing us with all the resources required to engage in this project.

I extend my special thanks to our Computer Science teacher Ms. Sonia Chacko ma'am for her constant guidance and support for the completion of my project.

I extend my gratitude towards all teaching and non-teaching staff of Chrysalis High.

Last but not the least, I'm forever indebted to my parents and friends for their support and encouragement throughout the spadework of this project.

Index

Sr. No.	Topic	Page No.
1.	Abstract	5
2.	Applications and Tools Used	7
3.	System Software and Hardware	10
4.	Structure of the Project	11
5.	Built-in Functions Used	14
6.	Project Flow of Control	17
7.	Source Code	19
8.	Output Screenshots	72
9.	Conclusion	78
10.	Limitations	79
11.	Bibliography	80

Abstract

BRIEF DESCRIPTION :

The motive of this project is to create an engaging, program-driven arcade game that is both entertaining and demonstrates core programming principles, including database management and GUI development.

PROJECT OVERVIEW :

- DinoJump is a side-scrolling infinite runner game inspired by the classic Chrome Dino game.
- The player controls a Dinosaur character that must jump to avoid oncoming obstacles (cacti and birds).
- The game features a user authentication system, global leaderboards, and progressive difficulty.
- It includes a modern, monochrome minimalist design for an enhanced user experience.

WHY I CHOSE THIS PROJECT :

- It provides a hands-on opportunity to integrate multiple programming concepts: GUI development with Tkinter, database management with MySQL, and game logic.
- The project demonstrates real-world application development with user accounts, data persistence, and a competitive scoring system.
- It serves as an excellent demonstration of object-oriented programming in Python.

MODULES USED :

- **USER AUTHENTICATION MODULE:** Handles user signup, login, and session management.
- **GAME ENGINE MODULE:** Manages the core game of mechanics, physics, and rendering.
- **LEADERBOARD MODULE:** Tracks and displays global and country-specific high scores.
- **DATABASE MODULE:** Stores user credentials, high scores, and leaderboard data persistently.

Applications and **Tools Used**

PYTHON:

Python is a high-level, general-purpose programming language. It is an interpreted language that emphasizes code readability with significant indentation. It is an object-oriented language that helps programmers write clear and logical code for both small and large projects.

The major features of Python are –

- (i) It is an interpreted language. Hence, debugging is easier.
- (ii) It is platform-independent and hence can be used on any device and is compatible with all major operating systems.
- (iii) It is completely free of cost to download. It is also an open-source software.

(iv) It is very simple to use since the language resembles the English language closely. Hence, it is very easy to learn how to write the commands and expressions.

(v) It also has a rich library support such as the Python Standard Library that provides a variety of functions for use.

TKINTER:

Tkinter is Python's de-facto standard GUI (Graphical User Interface) package. It is the most commonly used method for creating graphical applications in Python due to its simplicity and availability with standard Python installations.

Its unique features/characteristics are –

(i) Easy to Learn: Tkinter provides a simple and intuitive interface for creating GUI elements.

(ii) Cross-Platform: Tkinter applications can run on Windows, macOS, and Linux without modification.

(iii) Widget Variety: It offers a wide range of widgets like buttons, canvases, labels, and entry fields that can be used to build interactive applications.

(iv) Canvas for Graphics: The Canvas widget in Tkinter allows for drawing shapes, images, and creating animations, making it suitable for simple game development.

STRUCTURED QUERY LANGUAGE (SQL):

SQL (Structured Query Language) is a standardized language developed and used for accessing and modifying relational databases. The uses of SQL include modifying database table and index structures; adding, updating and deleting rows of data; and retrieving subsets of information from within a database.

Its unique features/characteristics are –

(i) It can be installed on various devices and is supported by a variety of operating systems.

(ii) It is highly secure with password protection and also has the feature of providing excellent connectivity. SQL can be used as a back-end for projects and such connectivity can be established with Python.

(iii) It is easy to use since it is a query-based language.

System Software and **Hardware Used**

SOFTWARE USED TO RUN THE PROJECT-

1. Windows 10.0 / macOS / Linux
2. Python IDLE, Visual Studio Code, PyCharm
3. Front end: Tkinter (Python)
4. Back end: MySQL (Structured Query Language)
5. MySQL Connector/Python

HARDWARE USED TO RUN THE PROJECT-

1. 1.10 GHz Dual-Core Processor or higher
2. 4/8 GB RAM
3. 250 GB HDD
4. Standard Keyboard for input

Structure of the Project

The project is designed with a multi-frame architecture, where each frame represents a different screen of the application. The main components are:

I. DATABASE LAYER:

- **MySQL Database:** Stores all persistent data.
- **Tables:**
 - **users:** Stores user credentials, country, and personal high score.
 - **global_leaderboard:** Stores all high scores for the global ranking system.

II. APPLICATION LAYER:

- **DinoJumpApp Class:** The main controller that manages navigation between frames and user sessions.
- **Frame Classes:**
 - **HomePageFrame:** The landing page with game description and play button.
 - **LoginFrame:** Handles user authentication (login/signup).
 - **HomeFrame:** The main menu showing user stats, leaderboards, and game start button.
 - **GameFrame:** Contains the actual game canvas and game logic.

III. GAME ENGINE:

- **Canvas-based Rendering:** Uses Tkinter's Canvas widget for all game graphics.
- **Game Loop:** Implemented using Tkinter's `after()` method for smooth animation.
- **Physics System:** Handles jumping mechanics with gravity and velocity.
- **Collision Detection:** Uses hitbox calculations for precise collision detection between dino and obstacles.

- **Progressive Difficulty:** Game speed and obstacle spawn rate increase with score.

Built-in Functions Used

TKINTER METHODS USED:

- a. **Tk()**: Creates the main application window.
- b. **Frame()**: Acts as a container for organizing widgets.
- c. **Canvas()**: Creates a drawing area for game graphics.
- d. **Label()**: Displays text such as scores and messages.
- e. **Button()**: Creates interactive buttons.
- f. **Entry()**: Creates text input fields for user credentials.
- g. **pack()**, **grid()**: Geometry managers for widget placement.
- h. **bind()**: Captures keyboard events for game controls.
- i. **after()**: Creates timed loops for game animation.

TKINTER CANVAS METHODS:

- a. **create_polygon()**: Draws the dinosaur and obstacle shapes.
- b. **create_text()**: Displays text directly on the canvas.

- c. **create_line()**: Draws the ground line.
- d. **coords()**: Gets or sets coordinates of canvas items.
- e. **move()**: Animates canvas items by moving them.
- f. **delete()**: Removes items from the canvas.

MYSQL CONNECTOR METHODS:

- a. **connect()**: Establishes connection to MySQL database.
- b. **cursor()**: Creates a cursor object for executing SQL queries.
- c. **execute()**: Executes database operations (INSERT, SELECT, UPDATE).
- d. **commit()**: Saves changes to the database.
- e. **fetchone(), fetchall()**: Retrieves rows from query results.

OTHER PYTHON MODULES:

- a. **random**: For generating random obstacles.
- b. **time**: For tracking game time and delta time calculations.

c. operator: For sorting and data manipulation.

Project Flow of **Control**

Start Program



Initialize Database & Tables



Show HomePageFrame



(Click PLAY NOW)

Show LoginFrame



(Successful Auth)

Show HomeFrame



(Click START GAME)

Show GameFrame



| GAME LOOP: |

| - Handle Input |

| - Update Physics |

| - Spawn Obstacles|

| - Check Collision|

| - Render Frame |



(Collision Detected)

Game Over



(Press Space)

Reset Game or Return to HomeFrame

Source Code

```
import tkinter as tk  
  
from tkinter import messagebox  
  
import random  
  
import time from operator  
  
import itemgetter  
  
import mysql.connector from mysql.connector  
  
import Error
```

--- MySQL Database Configuration ---

```
DB_CONFIG = { 'host': 'localhost', 'user': 'root', 'password':  
'root@123', 'database': 'dinojump_db' }  
  
def get_db_connection(): """Create and return a database  
connection.""" try: connection =  
mysql.connector.connect(**DB_CONFIG) return connection  
except Error as e: print(f'Error connecting to MySQL: {e}')  
return None
```

```

def init_database(): """Initialize database tables if they don't
exist."""
    connection = get_db_connection()
    if connection:
        try:
            cursor = connection.cursor()

            # Create users table if not exists
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INT AUTO_INCREMENT PRIMARY
KEY,
                    email VARCHAR(255) UNIQUE NOT
NULL,
                    password VARCHAR(255) NOT NULL,
                    country VARCHAR(10) DEFAULT
'INT',
                    high_score INT DEFAULT 0,
                    created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
                )
            """)

            # Create global_leaderboard table if
not exists
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS
global_leaderboard (
                    id INT AUTO_INCREMENT PRIMARY
KEY,

```

```

        username VARCHAR(255) NOT NULL,
        score INT NOT NULL,
        country VARCHAR(10) NOT NULL,
        created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
    )
    """)

    # Insert sample global leaderboard data
    if empty
        cursor.execute("SELECT COUNT(*) FROM
global_leaderboard")
        if cursor.fetchone()[0] == 0:
            sample_data = [
                ('NEO_MATRIX', 99999, 'US'),
                ('CYBER_GHOST', 15000, 'UK'),
                ('QUANTUM_LEAP', 12000, 'CA'),
                ('PHANTOM_BLADE', 10500,
'INT'),
                ('VOID_WALKER', 8000, 'JP')
            ]
            cursor.executemany(
                "INSERT INTO global_leaderboard
(username, score, country) VALUES
(%s, %s, %s)",
                sample_data

```

```

        )

        connection.commit()
        print("Database initialized
successfully")

    except Error as e:
        print(f"Error initializing database:
{e}")
    finally:
        cursor.close()
        connection.close()

```

Initialize database when module loads

```
init_database()
```

--- DinoJump Game Constants ---

```

GRAVITY = 0.5 JUMP_VELOCITY = -11
DINO_SIZE_FACTOR = 0.25 SCORE_MULTIPLIER = 0.12
GAME_START_SPEED = 8 GAME_MAX_SPEED = 16

```

--- Monochrome Nothing-Style Theme Colors

```
COLOR_PRIMARY = "#000000" COLOR_SECONDARY =  
"#a3a3a3" COLOR_BG = "#FFFFFF" COLOR_CARD_BG =  
"#f3f4f6" COLOR_TEXT = "#000000"
```

--- FONT Style Simulation ---

```
def get_font(size, style="bold"): return ("Courier", size, style)
```

Realistic gaming usernames for leaderboard

```
GAMER_NAMES = [ "NEO_MATRIX", "CYBER_GHOST",  
"QUANTUM_LEAP", "PHANTOM_BLADE",  
"VOID_WALKER", "SHADOW_STRIKE",  
"ECHO_SPECTRE", "BINARY_STORM",  
"CODE_BREAKER", "SYNTAX_ERROR" ]
```

```
def generate_large_leaderboard(count=1000): """Generates a  
large list of simulated high scores for fallback.""" leaderboard =  
[] countries = ["US", "UK", "CA", "INT", "JP", "DE", "FR",  
"AU", "BR", "IN", "CN"]
```

```
for i in range(count):  
    score = random.randint(100, 10000) +  
int((count - i) / count * 20000)  
    username = random.choice(GAMER_NAMES) +
```

```
f"_{random.randint(100, 999)}"    country =  
random.choice(countries)
```

```
    leaderboard.append({  
        "user": username,  
        "score": score,  
        "country": country  
    })
```

```
return leaderboard
```

```
FALLBACK_LEADERBOARD =  
generate_large_leaderboard(1000)
```

```
class DinoJumpApp(tk.Tk):  
    def init(self):  
        super().init()  
        self.title("DINOJUMP")  
        self.geometry("1000x700")  
        self.configure(bg=COLOR_BG)  
        self.resizable(True, True)  
  
        self.container = tk.Frame(self, bg=COLOR_BG)  
        self.container.pack(side="top",  
fill="both", expand=True)  
        self.container.grid_rowconfigure(0,  
weight=1)  
        self.container.grid_columnconfigure(0,  
weight=1)  
  
        self.current_user = None  
        self.current_score = 0
```



```

        self.frames = {}
        self.create_frames()
        self.show_frame("HomePage")

def create_frames(self):
    for F in (HomePageFrame, LoginFrame,
HomePageFrame, GameFrame):
        page_name = F.__name__.replace("Frame",
        "")
        frame = F(parent=self.container,
controller=self)
        self.frames[page_name] = frame
        frame.grid(row=0, column=0,
sticky="nsew")

def show_frame(self, page_name):
    frame = self.frames[page_name]
    frame.tkraise()

def handle_login(self, email, password):
    """Authenticate user with MySQL
database."""
    connection = get_db_connection()
    if not connection:
        return False

    try:

```

```

        cursor = connection.cursor()
        cursor.execute(
            "SELECT email, country, high_score
FROM users WHERE email = %s AND password = %s",
            (email, password)
        )
        result = cursor.fetchone()

        if result:
            self.current_user = email
            self.show_frame("Home")
            if hasattr(self.frames["Home"],
'update_user_info'):

self.frames["Home"].update_user_info()
            return True
        else:
            return False

    except Error as e:
        print(f"Login error: {e}")
        return False
    finally:
        cursor.close()
        connection.close()

def handle_signup(self, email, password):

```

```

        """Register new user in MySQL database."""
        # Validate email format
        if "@" not in email or "." not in
email.split("@")[-1]:
            return False, "ERROR: PLEASE ENTER A
VALID EMAIL ADDRESS."

        connection = get_db_connection()
        if not connection:
            return False, "ERROR: DATABASE
CONNECTION FAILED."

        try:
            cursor = connection.cursor()

            # Check if user already exists
            cursor.execute("SELECT email FROM users
WHERE email = %s", (email,))
            if cursor.fetchone():
                return False, "ERROR: USER ALREADY
EXISTS."

            # Insert new user
            countries = ["US", "UK", "CA", "INT"]
            country = random.choice(countries)

            cursor.execute(
                "INSERT INTO users (email,

```

```

password, country) VALUES (%s, %s, %s)",
    (email, password, country)
)
connection.commit()

self.current_user = email
self.show_frame("Home")
if hasattr(self.frames["Home"],
'update_user_info'):

self.frames["Home"].update_user_info()
    return True, "SIGN UP SUCCESSFUL!"

except Error as e:
    print(f"Signup error: {e}")
    return False, "ERROR: DATABASE ERROR.
PLEASE TRY AGAIN."
finally:
    cursor.close()
    connection.close()

def handle_logout(self):
    """Log out current user and clear login
fields."""
    self.current_user = None
    login_frame = self.frames["Login"]
    login_frame.email_entry.delete(0, tk.END)

```

```

        login_frame.password_entry.delete(0,
tk.END)
        login_frame.error_label.config(text="")
        self.show_frame("HomePage")

def get_high_score(self):
    """Retrieve high score from MySQL
database."""
    if not self.current_user:
        return 0

    connection = get_db_connection()
    if not connection:
        return 0

    try:
        cursor = connection.cursor()
        cursor.execute(
            "SELECT high_score FROM users WHERE
email = %s",
            (self.current_user,)
        )
        result = cursor.fetchone()
        return result[0] if result else 0
    except Error as e:
        print(f"Error getting high score: {e}")
        return 0

```

```

    finally:
        cursor.close()
        connection.close()

def update_high_score(self, new_score):
    """Update high score in MySQL database if
new score is higher."""
    if not self.current_user:
        return

    int_score = int(new_score)
    current_high_score = self.get_high_score()

    if int_score > current_high_score:
        connection = get_db_connection()
        if not connection:
            return

        try:
            cursor = connection.cursor()
            cursor.execute(
                "UPDATE users SET high_score
= %s WHERE email = %s",
                (int_score, self.current_user)
            )
            connection.commit()

            # Also add to global leaderboard

```

```

        username =
self.current_user.split('@')[0].upper()
        cursor.execute(
            "SELECT country FROM users
WHERE email = %s",
            (self.current_user,)
        )
        country_result = cursor.fetchone()
        country = country_result[0] if
country_result else 'INT'

        cursor.execute(
            "INSERT INTO global_leaderboard
(username, score, country) VALUES
(%s, %s, %s)",
            (username, int_score, country)
        )
        connection.commit()

    except Error as e:
        print(f"Error updating high score:
{e}")

    finally:
        cursor.close()
        connection.close()

def get_user_rank_and_leaderboards(self):

```

```

        """Calculate user rank and return formatted
        leaderboards from MySQL."""
        connection = get_db_connection()
        if not connection:
            return
self._get_fallback_leaderboards()

        try:
            cursor = connection.cursor()

            # Get global leaderboard
            cursor.execute("""
                SELECT username, score, country
                FROM global_leaderboard
                ORDER BY score DESC
                LIMIT 5
            """)
            global_lb = [{"user": row[0], "score":
row[1], "country": row[2]}
                        for row in
cursor.fetchall()]

            # Get user info if logged in
            user_global_rank = "N/A"
            user_country_rank = "N/A"
            user_country = "N/A"
            country_lb = []

```



```

    if self.current_user:
        # Get user's country
        cursor.execute("SELECT country FROM
users WHERE email = %s", (self.current_user,))
        country_result = cursor.fetchone()
        user_country = country_result[0] if
country_result else 'INT'

        # Get country leaderboard
        cursor.execute("""
            SELECT username, score, country
            FROM global_leaderboard
            WHERE country = %s
            ORDER BY score DESC
            LIMIT 5
            """, (user_country,))
        country_lb = [{"user": row[0],
"score": row[1], "country": row[2]}
                        for row in
cursor.fetchall()]

        # Calculate user's global rank
        cursor.execute("""
            SELECT COUNT(*) + 1
            FROM global_leaderboard
            WHERE score > (SELECT

```

```

high_score FROM users WHERE email = %s)
        """ , (self.current_user,))
        user_global_rank =
cursor.fetchone()[0]

        # Calculate user's country rank
        cursor.execute("""
            SELECT COUNT(*) + 1
            FROM global_leaderboard
            WHERE country = %s AND score >
(SELECT high_score FROM users WHERE email = %s)
            """ , (user_country,
self.current_user))
        user_country_rank =
cursor.fetchone()[0]

        return {
            "global_lb": global_lb,
            "country_lb": country_lb,
            "global_rank": user_global_rank,
            "country_rank": user_country_rank,
            "user_country": user_country
        }

    except Error as e:
        print(f"Error getting leaderboards:
{e}")

```

```

        return
self._get_fallback_leaderboards()
    finally:
        cursor.close()
        connection.close()

def _get_fallback_leaderboards(self):
    """Fallback leaderboard data if database is
    unavailable."""
    user_global_rank = "N/A"
    user_country_rank = "N/A"
    user_country = "N/A"

    if self.current_user:
        username =
self.current user.split('@')[0].upper()
        user_country = random.choice(["US",
        "UK", "CA", "INT"])

        # Simple rank calculation from fallback
        data

        all_scores = sorted([item['score'] for
        item in FALLBACK_LEADERBOARD], reverse=True)
        user_score = self.get_high_score()
        user_global_rank = next((i + 1 for i,
        score in enumerate(all_scores) if score <=

```

```

user_score), "N/A")

        country_scores = sorted([item['score']
for item in FALLBACK_LEADERBOARD if
item['country'] == user_country], reverse=True)
        user_country_rank = next((i + 1 for i,
score in enumerate(country_scores) if score <=
user_score), "N/A")

    return {
        "global_lb": FALLBACK_LEADERBOARD[:5],
        "country_lb": [item for item in
FALLBACK_LEADERBOARD if item['country'] ==
user_country][:5],
        "global_rank": user_global_rank,
        "country_rank": user_country_rank,
        "user_country": user_country
    }

```

```

class HomePageFrame(tk.Frame):
    def init(self, parent, controller):
        super().init(parent, bg=COLOR_BG)
        self.controller = controller

        self.grid_rowconfigure(0, weight=1)
        self.grid_columnconfigure(0, weight=1)

        main_container = tk.Frame(self,

```

```

bg=COLOR_BG)
    main_container.pack(fill="both",
expand=True, padx=50, pady=50)

    title_frame = tk.Frame(main_container,
bg=COLOR_BG)
    title_frame.pack(pady=(0, 40))

    tk.Label(title_frame, text="DINOJUMP",
font=get_font(48, "bold"),
fg=COLOR_PRIMARY,
bg=COLOR_BG).pack()

    tk.Label(title_frame, text="THE ULTIMATE
JUMPING EXPERIENCE", font=get_font(16),
fg=COLOR_SECONDARY,
bg=COLOR_BG).pack(pady=(10, 5))

    tk.Label(title_frame, text="Developed by
Venkata Sai Kushal Reddy Pujari (XII B)",
font=get_font(12),
fg=COLOR_SECONDARY, bg=COLOR_BG).pack()

    desc_card = tk.Frame(main_container,
bg=COLOR_CARD_BG, padx=40, pady=30,
relief="flat",
highlightbackground=COLOR_PRIMARY,
highlightthickness=2)

```

```

desc_card.pack(fill="x", pady=(0, 40))

description_text = (
    "WELCOME TO DINOJUMP - THE MOST
    THRILLING JUMP ADVENTURE!\n\n"
    "• GUIDE YOUR DINO THROUGH THE DESERT
    LANDSCAPE\n"
    "• JUMP OVER CACTI AND DODGE FLYING
    OBSTACLES\n"
    "• SURVIVE AS LONG AS POSSIBLE TO
    ACHIEVE HIGH SCORES\n"
    "• COMPETE WITH PLAYERS WORLDWIDE ON
    THE LEADERBOARDS\n\n"
    "FEATURES:\n"
    "✓ SMOOTH GAMEPLAY PHYSICS\n"
    "✓ PROGRESSIVE DIFFICULTY\n"
    "✓ GLOBAL RANKING SYSTEM\n"
    "✓ MONOCHROME MINIMALIST DESIGN"
)

tk.Label(desc_card, text=description_text,
font=get_font(12),
          fg=COLOR_TEXT, bg=COLOR_CARD_BG,
justify="center").pack()

play_button = tk.Button(main_container,

```

```

text="PLAY NOW",
                                command=lambda:
controller.show_frame("Login"),
                                font=get_font(24,
"bold"), fg="#FFFFFF", bg=COLOR_PRIMARY,

activebackground=COLOR_SECONDARY,
activeforeground="#FFFFFF",
                                bd=0,
relief="raised", padx=50, pady=20,
cursor="hand2")
    play_button.pack(pady=(0, 30))

    footer_frame = tk.Frame(main_container,
bg=COLOR_BG)
    footer_frame.pack(side="bottom", fill="x")
tk.Label(footer_frame, text="CHALLENGE YOUR
REFLEXES • BEAT YOUR RECORDS • BECOME
LEGENDARY",
                                font=get_font(10),
fg=COLOR_SECONDARY, bg=COLOR_BG).pack(pady=10)

class LoginFrame(tk.Frame): def init(self, parent, controller):
super().init(parent, bg=COLOR_BG) self.controller = controller

```

```

        self.grid_rowconfigure(0, weight=1)
        self.grid_columnconfigure(0, weight=1)

        self.is_login_mode =
tk.BooleanVar(value=True)

        main_container = tk.Frame(self,
bg=COLOR_BG)
        main_container.pack(fill="both",
expand=True, padx=50, pady=50)

        header = tk.Frame(main_container,
bg=COLOR_BG)
        header.pack(fill="x", pady=(0, 20))

        tk.Button(header, text="← BACK TO HOME",
                    command=lambda:
controller.show_frame("HomePage"),
                    font=get_font(10, "bold"),
fg="#FFFFFF", bg=COLOR_SECONDARY,
                    bd=0, relief="raised", padx=15,
pady=5).pack(side="left")

        card_container = tk.Frame(main_container,
bg=COLOR_BG)
        card_container.pack(expand=True)

        card = tk.Frame(card_container,

```



```

bg=COLOR_CARD_BG, padx=50, pady=40, bd=0,
relief="flat",

highlightbackground=COLOR_TEXT,
highlightthickness=2)
    card.pack()

    tk.Label(card, text="DINOJUMP ACCESS",
font=get_font(24, "bold"),
            fg=COLOR_TEXT,
bg=COLOR_CARD_BG).pack(pady=(0, 30))

    self.error_label = tk.Label(card, text="",
font=get_font(10), fg="#ff0000",
bg=COLOR_CARD_BG)
    self.error_label.pack(fill="x", pady=(0,
10))

    self.email_label = tk.Label(card,
text="EMAIL ADDRESS:", font=get_font(12),
                                fg=COLOR_TEXT,
bg=COLOR_CARD_BG, anchor="w")
    self.email_label.pack(fill="x", pady=(10,
5))

    self.email_entry = tk.Entry(card,
font=get_font(12), bd=1, relief="flat",

```

```

highlightthickness=1,
highlightbackground=COLOR_SECONDARY,
                                                    fg=COLOR_TEXT,
insertbackground=COLOR_TEXT)
        self.email_entry.pack(fill="x", ipady=6,
pady=(0, 15))

        self.password_label = tk.Label(card,
text="PASSWORD:", font=get_font(12),

fg=COLOR_TEXT, bg=COLOR_CARD_BG, anchor="w")
        self.password_label.pack(fill="x",
pady=(10, 5))

        self.password_entry = tk.Entry(card,
font=get_font(12), show="*", bd=1,
relief="flat",

highlightthickness=1,
highlightbackground=COLOR_SECONDARY,

fg=COLOR_TEXT, insertbackground=COLOR_TEXT)
        self.password_entry.pack(fill="x", ipady=6,
pady=(0, 15))

        self.action_button = tk.Button(card,

```

```

text="LOGIN", command=self._handle_action,

font=get_font(14, "bold"), fg="#FFFFFF",
bg=COLOR_PRIMARY,

activebackground=COLOR_SECONDARY,
activeforeground="#FFFFFF",

                                bd=0,
relief="raised", padx=10, pady=8)
    self.action_button.pack(fill="x", pady=(20,
15))

    self.switch_link = tk.Label(card,
text="NEED AN ACCOUNT? SIGN UP",

font=get_font(10, "underline"), fg=COLOR_TEXT,
bg=COLOR_CARD_BG,

                                cursor="hand2")
    self.switch_link.bind("<Button-1>",
self._toggle_mode)
    self.switch_link.pack(fill="x")

def _toggle_mode(self, event=None):
    self.is_login_mode.set(not
self.is_login_mode.get())
    if self.is_login_mode.get():
        self.action_button.config(text="LOGIN")

```

```

        self.switch_link.config(text="NEED AN
ACCOUNT? SIGN UP")
    else:
        self.action_button.config(text="SIGN
UP")
        self.switch_link.config(text="HAVE AN
ACCOUNT? LOGIN")
        self.error_label.config(text="")

def _handle_action(self):
    email = self.email_entry.get()
    password = self.password_entry.get()
    self.error_label.config(text="")

    if not email or not password:
        self.error_label.config(text="EMAIL AND
PASSWORD ARE REQUIRED.", fg="#ff0000")
        return

    if self.is_login_mode.get():
        if self.controller.handle_login(email,
password):
            pass
        else:

self.error_label.config(text="INVALID EMAIL OR
PASSWORD.", fg="#ff0000")

```

```

        else:
            success, message =
self.controller.handle_signup(email, password)
            if not success:

self.error_label.config(text=message,
fg="#ff0000")

class HomeFrame(tk.Frame): def init(self, parent, controller):
super().init(parent, bg=COLOR_BG) self.controller = controller
self.grid_rowconfigure(0, weight=1)
    self.grid_columnconfigure(0, weight=1)

    main_container = tk.Frame(self,
bg=COLOR_BG)
    main_container.pack(fill="both",
expand=True, padx=20, pady=20)

    self.header = tk.Frame(main_container,
bg=COLOR_CARD_BG, padx=20, pady=15,
                                relief="flat",
highlightbackground=COLOR_SECONDARY,
highlightthickness=1)
    self.header.pack(fill="x", pady=(0, 20))

    tk.Label(self.header, text="DINOJUMP",
font=get_font(20, "bold"),

```

```

        fg=COLOR_PRIMARY,
bg=COLOR_CARD_BG).pack(side="left")

        tk.Button(self.header, text="LOGOUT",
command=controller.handle_logout,
                font=get_font(10, "bold"),
fg="#FFFFFF", bg=COLOR_SECONDARY,
                bd=0, relief="raised", padx=15,
pady=5).pack(side="right")

        welcome_frame = tk.Frame(main_container,
bg=COLOR_BG)
        welcome_frame.pack(fill="x", pady=(0, 20))

        self.welcome_label =
tk.Label(welcome_frame, text="WELCOME BACK!",

font=get_font(14, "italic"), fg=COLOR_TEXT,
bg=COLOR_BG)
        self.welcome_label.pack()

        button_frame = tk.Frame(main_container,
bg=COLOR_BG)
        button_frame.pack(fill="x", pady=(0, 30))

        tk.Button(button_frame, text="START GAME",
command=self._start_game,
                font=get_font(20, "bold"),

```

```
fg="#FFFFFF", bg=COLOR_PRIMARY,  
            activebackground=COLOR_SECONDARY,  
activeforeground="#FFFFFF",  
            bd=0, relief="raised", padx=40,  
pady=15).pack()
```

```
    content_frame = tk.Frame(main_container,  
bg=COLOR_BG)  
    content_frame.pack(fill="both",  
expand=True)
```

```
    self.user_score_card =  
self._create_user_stats(content_frame)  
    self.user_score_card.pack(side="left",  
fill="both", expand=True, padx=10)
```

```
    self.global_lb_frame =  
self._create_leaderboard_panel(content_frame,  
"GLOBAL TOP 5", "global")  
    self.global_lb_frame.pack(side="left",  
fill="both", expand=True, padx=10)
```

```
    self.country_lb_frame =  
self._create_leaderboard_panel(content_frame,  
"COUNTRY TOP 5", "country")  
    self.country_lb_frame.pack(side="left",  
fill="both", expand=True, padx=10)
```

```

def _create_user_stats(self, parent):
    card = tk.Frame(parent, bg=COLOR_CARD_BG,
        padx=25, pady=20, bd=0, relief="flat",

        highlightbackground=COLOR_PRIMARY,
        highlightthickness=2)

    tk.Label(card, text="MY STATS",
        font=get_font(14, "bold"),
        fg=COLOR_TEXT,
        bg=COLOR_CARD_BG).pack(pady=(0, 20))

    stats_frame = tk.Frame(card,
        bg=COLOR_CARD_BG)
    stats_frame.pack(fill="both", expand=True)

    score_frame = tk.Frame(stats_frame,
        bg=COLOR_CARD_BG)
    score_frame.pack(fill="x", pady=10)
    tk.Label(score_frame, text="HIGH SCORE:",
        font=get_font(11),
        fg=COLOR_TEXT,
        bg=COLOR_CARD_BG).pack(side="left")
    self.my_score_label = tk.Label(score_frame,
        text="0", font=get_font(18, "bold"),

        fg=COLOR_PRIMARY, bg=COLOR_CARD_BG)

```



```

        self.my_score_label.pack(side="right")

        global_frame = tk.Frame(stats_frame,
bg=COLOR_CARD_BG)
        global_frame.pack(fill="x", pady=10)
        tk.Label(global_frame, text="GLOBAL RANK:",
font=get_font(11),
                fg=COLOR_TEXT,
bg=COLOR_CARD_BG).pack(side="left")
        self.global_rank_label =
tk.Label(global_frame, text="N/A",
font=get_font(18, "bold"),

fg=COLOR_PRIMARY, bg=COLOR_CARD_BG)
        self.global_rank_label.pack(side="right")

        country_frame = tk.Frame(stats_frame,
bg=COLOR_CARD_BG)
        country_frame.pack(fill="x", pady=10)
        tk.Label(country_frame, text="COUNTRY
RANK:", font=get_font(11),
                fg=COLOR_TEXT,
bg=COLOR_CARD_BG).pack(side="left")
        self.country_rank_label =
tk.Label(country_frame, text="N/A",
font=get_font(18, "bold"),

```

```

fg=COLOR_PRIMARY, bg=COLOR_CARD_BG)
    self.country_rank_label.pack(side="right")

    self.user_country_label = tk.Label(card,
text="(COUNTRY: LOADING...)",

font=get_font(9), fg=COLOR_TEXT,
bg=COLOR_CARD_BG)
    self.user_country_label.pack(side="bottom",
pady=(20, 0), anchor="e")

    return card

def _create_leaderboard_panel(self, parent,
title, ref_name):
    frame = tk.Frame(parent, bg=COLOR_CARD_BG,
padx=20, pady=15, bd=0, relief="flat",

highlightbackground=COLOR_SECONDARY,
highlightthickness=1)

    tk.Label(frame, text=title,
font=get_font(14, "bold"),
fg=COLOR_PRIMARY,
bg=COLOR_CARD_BG).pack(pady=(0, 15))

    header_frame = tk.Frame(frame,
bg=COLOR_CARD_BG)

```

```

        header_frame.pack(fill="x", pady=(0, 10))

        tk.Label(header_frame, text="RANK",
font=get_font(10, "bold"),
                fg=COLOR_TEXT, bg=COLOR_CARD_BG,
width=6, anchor="w").pack(side="left")
        tk.Label(header_frame, text="USER",
font=get_font(10, "bold"),
                fg=COLOR_TEXT,
bg=COLOR_CARD_BG).pack(side="left", fill="x",
expand=True)
        tk.Label(header_frame, text="SCORE",
font=get_font(10, "bold"),
                fg=COLOR_TEXT, bg=COLOR_CARD_BG,
width=8, anchor="e").pack(side="right")

        entries_frame = tk.Frame(frame,
bg=COLOR_CARD_BG)
        entries_frame.pack(fill="both",
expand=True)

        for i in range(5):
            row_frame = tk.Frame(entries_frame,
bg=COLOR_CARD_BG)
            row_frame.pack(fill="x", pady=5)

            rank_label = tk.Label(row_frame,

```

```

text=f"{i+1}.", font=get_font(11),
                                fg=COLOR_TEXT,
bg=COLOR_CARD_BG, width=6, anchor="w")
    rank_label.pack(side="left")

    user_label = tk.Label(row_frame,
text="---", font=get_font(11),
                                fg=COLOR_TEXT,
bg=COLOR_CARD_BG, anchor="w")
    user_label.pack(side="left", fill="x",
expand=True)

    score_label = tk.Label(row_frame,
text="---", font=get_font(11),
                                fg=COLOR_TEXT,
bg=COLOR_CARD_BG, width=8, anchor="e")
    score_label.pack(side="right")

    setattr(self, f"{ref_name}_rank_{i}",
rank_label)
    setattr(self, f"{ref_name}_user_{i}",
user_label)
    setattr(self, f"{ref_name}_score_{i}",
score_label)

    return frame

def update_user_info(self):

```

```

        user = self.controller.current_user if
self.controller.current_user else None

        display_user = user.split('@')[0].upper()
if user else "GUEST"
        self.welcome_label.config(text=f"WELCOME
BACK, {display_user}!")

        lb_data =
self.controller.get_user_rank_and_leaderboards(
)

self.my_score_label.config(text=str(self.contro
ller.get_high_score()))

self.global_rank_label.config(text=str(lb_data[
'global_rank']))

self.country_rank_label.config(text=str(lb_data
['country_rank']))

self.user_country_label.config(text=f"(COUNTRY:
{lb_data['user_country']})" if user else
"(GUEST MODE)")

```

```

self._update_leaderboard(lb_data['global_lb'],
"global")

self._update_leaderboard(lb_data['country_lb'],
"country")

def _update_leaderboard(self, data_list,
prefix):

    for i in range(5):
        user_label = getattr(self,
f"{prefix}_user_{i}")
        score_label = getattr(self,
f"{prefix}_score_{i}")

        if i < len(data_list):
            item = data_list[i]

            user_label.config(text=item['user'])

            score_label.config(text=str(item['score']))
        else:
            user_label.config(text="---")
            score_label.config(text="---")

def _start_game(self):
    self.controller.show_frame("Game")

```

```

        self.controller.frames["Game"].reset_game()

class GameFrame(tk.Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg=COLOR_BG)
        self.controller = controller

        main_container = tk.Frame(self, bg=COLOR_BG)
        main_container.pack(fill="both",
                           expand=True, padx=20, pady=20)

        header = tk.Frame(main_container,
                           bg=COLOR_CARD_BG, padx=20, pady=10,
                           relief="flat",

                           highlightbackground=COLOR_SECONDARY,
                           highlightthickness=1)
        header.pack(fill="x", pady=(0, 20))

        self.score_label = tk.Label(header,
                                     text="SCORE: 00000 | HI: 00000",

                                     font=get_font(14, "bold"), fg=COLOR_TEXT,
                                     bg=COLOR_CARD_BG)
        self.score_label.pack(side="left")

        tk.Button(header, text="BACK TO HOME",
                  command=lambda:
                    (self.controller.show_frame("Home")),

```

```

self.controller.frames["Home"].update_user_info
()),
            font=get_font(10, "bold"),
fg="#FFFFFF", bg=COLOR_SECONDARY,
            bd=0, relief="raised", padx=15,
pady=5).pack(side="right")

        canvas_container = tk.Frame(main_container,
bg=COLOR_BG)
        canvas_container.pack(fill="both",
expand=True)

        self.canvas_width = 900
        self.canvas_height = 300

        self.canvas = tk.Canvas(canvas_container,
width=self.canvas_width,
height=self.canvas_height,
                                bg=COLOR_BG, bd=0,
highlightthickness=2,
highlightbackground=COLOR_PRIMARY,
                                relief="flat")
        self.canvas.pack(expand=True)

        self.is_running = False
        self.dino = {}
        self.obstacles = []

```



```

        self._raw_distance = 0
        self.score = 0
        self.game_speed = GAME_START_SPEED
        self.time_since_last_obstacle = 0
        self.base_obstacle_spawn_rate = 1500
        self.obstacle_spawn_rate =
self.base_obstacle_spawn_rate
        self.last_frame_time = time.time()
        self.waiting_for_restart = False

        self.draw_ground()
        self.draw_dino_initial()
        self.draw_game_over_message("PRESS SPACE TO
START")

        self.controller.bind("<space>",
self.on_key_press)
        self.controller.bind("<Up>",
self.on_key_press)

@property
def dino_size(self):
    return max(30, int(self.canvas_height *
DINO_SIZE_FACTOR))

def draw_ground(self):
    self.canvas.create_line(0,

```

```

self.canvas_height - 2, self.canvas_width,
self.canvas_height - 2,
                                fill=COLOR_PRIMARY,
width=2, tags="ground")

def draw_dino_initial(self):
    size = self.dino_size
    x_start = size
    y_ground = self.canvas_height - 2

    points = [
        x_start, y_ground,
        x_start + size * 0.3, y_ground,
        x_start + size * 0.3, y_ground - size *
0.4,
        x_start + size * 0.5, y_ground - size *
0.4,
        x_start + size * 0.5, y_ground - size *
0.8,
        x_start + size * 0.2, y_ground - size *
0.9,
        x_start + size * 0.4, y_ground - size *
0.7,
        x_start + size * 0.8, y_ground - size *
0.7,
        x_start + size * 0.9, y_ground - size *
0.6,

```

```

        x_start + size * 0.8, y_ground - size *
0.5,
        x_start + size * 0.6, y_ground - size *
0.5,
        x_start + size * 0.6, y_ground - size *
0.3,
        x_start + size * 0.4, y_ground - size *
0.3,
        x_start + size * 0.4, y_ground,
        x_start + size * 0.6, y_ground,
        x_start + size * 0.6, y_ground - size *
0.2,
        x_start + size * 0.5, y_ground - size *
0.2,
        x_start + size * 0.5, y_ground - size *
0.4,
        x_start + size * 0.3, y_ground - size *
0.4,
        x_start + size * 0.3, y_ground,
        x_start, y_ground
    ]

```

```

self.canvas.delete("dino")

```

```

self.dino = {
    "x": x_start,
    "y": y_ground - size,
}

```

```

        "dy": 0,
        "is_jumping": False,
        "id":
self.canvas.create_polygon(points,
fill=COLOR_PRIMARY, outline=COLOR_PRIMARY,
tags="dino"),
        "size": size,
        "ground_y": y_ground,
        "hitbox": None
    }

def draw_game_over_message(self, message):
    self.canvas.delete("message")
    self.canvas.create_text(self.canvas_width
// 2, self.canvas_height // 2,
                            text=message,
font=get_font(20, "bold"),
                            fill=COLOR_PRIMARY,
tags="message")

def on_key_press(self, event):
    if event.keysym in ("space", "Up"):
        if not self.is_running and not
self.waiting_for_restart:
            self.start_game()
        elif self.waiting_for_restart:
            self.reset_game()

```

```

        else:
            self.jump()

def jump(self):
    if not self.dino["is_jumping"]:
        self.dino["dy"] = JUMP_VELOCITY
        self.dino["is_jumping"] = True

def start_game(self):
    self.is_running = True
    self.waiting_for_restart = False
    self.score = 0
    self._raw_distance = 0
    self.game_speed = GAME_START_SPEED
    self.obstacle_spawn_rate =
self.base_obstacle_spawn_rate
    self.obstacles = []
    self.time_since_last_obstacle = 0
    self.canvas.delete("message")
    self.canvas.delete("obstacle")

    self.draw_dino_initial()
    self.update_score_display()
    self.game_loop()

def reset_game(self):
    self.is_running = False

```

```

self.waiting_for_restart = False
self.score = 0
self._raw_distance = 0
self.game_speed = GAME_START_SPEED
self.obstacles = []
self.time_since_last_obstacle = 0
self.canvas.delete("obstacle")
self.canvas.delete("message")

self.draw_dino_initial()
self.draw_game_over_message("PRESS SPACE TO
START")
self.update_score_display()

def update_score_display(self):
    hi_score = self.controller.get_high_score()
    self.score_label.config(text=f"SCORE:
{self.score:05d} | HI: {hi_score:05d}")

def spawn_obstacle(self):
    obstacle_types = [
        {"type": "cactus", "width":
self.dino_size * 0.4, "height": self.dino_size
* 0.8},
        {"type": "cactus", "width":
self.dino_size * 0.6, "height": self.dino_size
* 0.7},

```

```
        {"type": "cactus", "width":  
self.dino_size * 0.3, "height": self.dino_size  
* 0.9},  
        {"type": "bird", "width":  
self.dino_size * 0.8, "height": self.dino_size  
* 0.4}  
    ]
```

```
    obstacle_data =  
random.choice(obstacle_types)
```

```
    if obstacle_data["type"] == "bird":  
        height_variation = random.choice([0.3,  
0.5, 0.7])  
        y_pos = self.dino["ground_y"] -  
obstacle_data["height"] - (self.dino_size *  
height_variation)  
    else:  
        y_pos = self.dino["ground_y"] -  
obstacle_data["height"]
```

```
    obstacle = {  
        "type": obstacle_data["type"],  
        "x": self.canvas_width,  
        "y": y_pos,  
        "width": obstacle_data["width"],  
        "height": obstacle_data["height"],
```

```

        "id":
self.create_obstacle_shape(obstacle_data["type"
], self.canvas_width, y_pos,

obstacle_data["width"],
obstacle_data["height"]),
        "hitbox": None
    }
    self.obstacles.append(obstacle)

def create_obstacle_shape(self, obs_type, x, y,
width, height):
    if obs_type == "cactus":
        points = [
            x, y + height,
            x + width * 0.3, y + height,
            x + width * 0.3, y + height * 0.2,
            x + width * 0.5, y + height * 0.2,
            x + width * 0.5, y,
            x + width * 0.3, y,
            x + width * 0.3, y + height * 0.4,
            x + width * 0.1, y + height * 0.4,
            x + width * 0.1, y + height * 0.2,
            x, y + height * 0.2,
            x, y + height
        ]

```



```

        return
self.canvas.create_polygon(points,
fill=COLOR_PRIMARY, outline=COLOR_PRIMARY,
tags="obstacle")
    else:
        points = [
            x, y + height / 2,
            x + width, y,
            x + width, y + height,
            x, y + height / 2
        ]
        return
self.canvas.create_polygon(points,
fill=COLOR_PRIMARY, outline=COLOR_PRIMARY,
tags="obstacle")

def update_dino(self, delta_time):
    self.dino["dy"] += GRAVITY

    dino_coords =
self.canvas.coords(self.dino["id"])
    if not dino_coords:
        return

    y_coords = dino_coords[1::2]
    current_y = min(y_coords)

```

```

        new_y = current_y + self.dino["dy"]
        ground_y = self.dino["ground_y"] -
self.dino_size

        if new_y >= ground_y:
            new_y = ground_y
            self.dino["dy"] = 0
            self.dino["is_jumping"] = False

        self.canvas.move(self.dino["id"], 0, new_y
- current_y)
        self._update_dino_hitbox()

def _update_dino_hitbox(self):
    dino_coords =
self.canvas.coords(self.dino["id"])
    if not dino_coords:
        return

    x_coords = dino_coords[0::2]
    y_coords = dino_coords[1::2]

    padding_x = 5
    padding_y = 8

    min_x = min(x_coords) + padding_x
    max_x = max(x_coords) - padding_x
    min_y = min(y_coords) + padding_y

```

```

        max_y = max(y_coords) - padding_y

        self.dino["hitbox"] = (min_x, min_y, max_x,
max_y)

def _update_obstacle_hitbox(self, obstacle):
    obs_coords =
self.canvas.coords(obstacle["id"])
    if not obs_coords:
        return

    x_coords = obs_coords[0::2]
    y_coords = obs_coords[1::2]

    padding_x = 3
    padding_y = 3

    min_x = min(x_coords) + padding_x
    max_x = max(x_coords) - padding_x
    min_y = min(y_coords) + padding_y
    max_y = max(y_coords) - padding_y

    obstacle["hitbox"] = (min_x, min_y, max_x,
max_y)

def update_obstacles(self, delta_time):
    speed_pixels = self.game_speed * delta_time
* 60

```

```

    obstacles_to_remove = []

    for obstacle in self.obstacles:
        self.canvas.move(obstacle["id"], -
speed_pixels, 0)
        obstacle["x"] -= speed_pixels

        self._update_obstacle_hitbox(obstacle)

        if obstacle["x"] + obstacle["width"] <
0:

obstacles_to_remove.append(obstacle)
            continue

        if self.check_collision(obstacle):
            self.game_over()
            return

    for obstacle in obstacles_to_remove:
        self.obstacles.remove(obstacle)
        self.canvas.delete(obstacle["id"])

def check_collision(self, obstacle):
    if not self.dino["hitbox"] or not
obstacle["hitbox"]:
        return False

```

```

        dino_hitbox = self.dino["hitbox"]
        obs_hitbox = obstacle["hitbox"]

        collision = (dino_hitbox[0] < obs_hitbox[2]
and
                    dino_hitbox[2] > obs_hitbox[0]
and
                    dino_hitbox[1] < obs_hitbox[3]
and
                    dino_hitbox[3] > obs_hitbox[1])

        return collision

def update_game_speed(self):
    self.game_speed = min(GAME_MAX_SPEED,
GAME_START_SPEED + (self.score * 0.001))
    self.obstacle_spawn_rate = max(500,
self.base_obstacle_spawn_rate - (self.score *
0.1))

def game_over(self):
    self.is_running = False
    self.waiting_for_restart = True

self.controller.update_high_score(self.score)
    self.draw_game_over_message("GAME
OVER\nPRESS SPACE TO RESTART")

```

```

def game_loop(self):
    if not self.is_running:
        return

    current_time = time.time()
    delta_time = current_time -
self.last_frame_time
    self.last_frame_time = current_time

    self._raw_distance += self.game_speed *
delta_time
    self.score = int(self._raw_distance *
SCORE_MULTIPLIER)
    self.update_score_display()
    self.update_game_speed()

    self.update_dino(delta_time)

    self.time_since_last_obstacle += delta_time
* 1000
    if self.time_since_last_obstacle >
self.obstacle_spawn_rate:
        self.spawn_obstacle()
        self.time_since_last_obstacle = 0

    self.update_obstacles(delta_time)

```

```
if self.is_running:  
    self.after(16, self.game_loop)
```

--- APPLICATION STARTUP ---

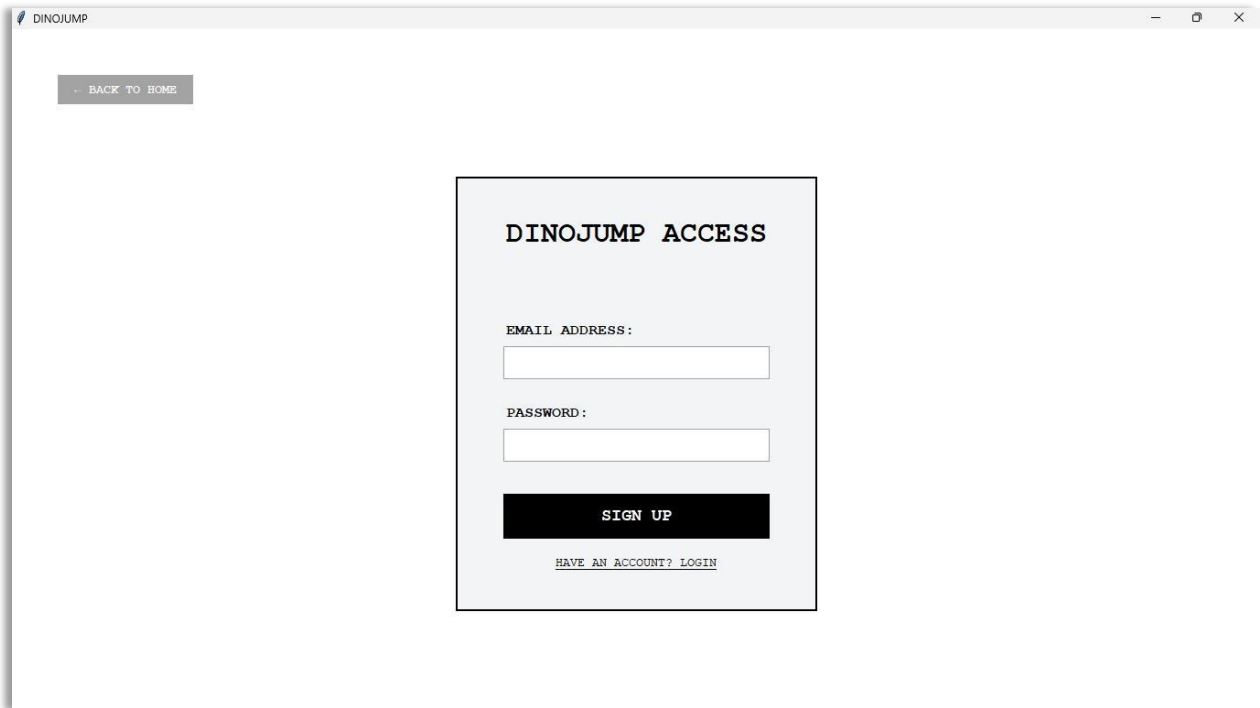
```
if name == "main":  
  
    app = DinoJumpApp()  
  
    app.mainloop()
```

Output Screenshots

Introduction:

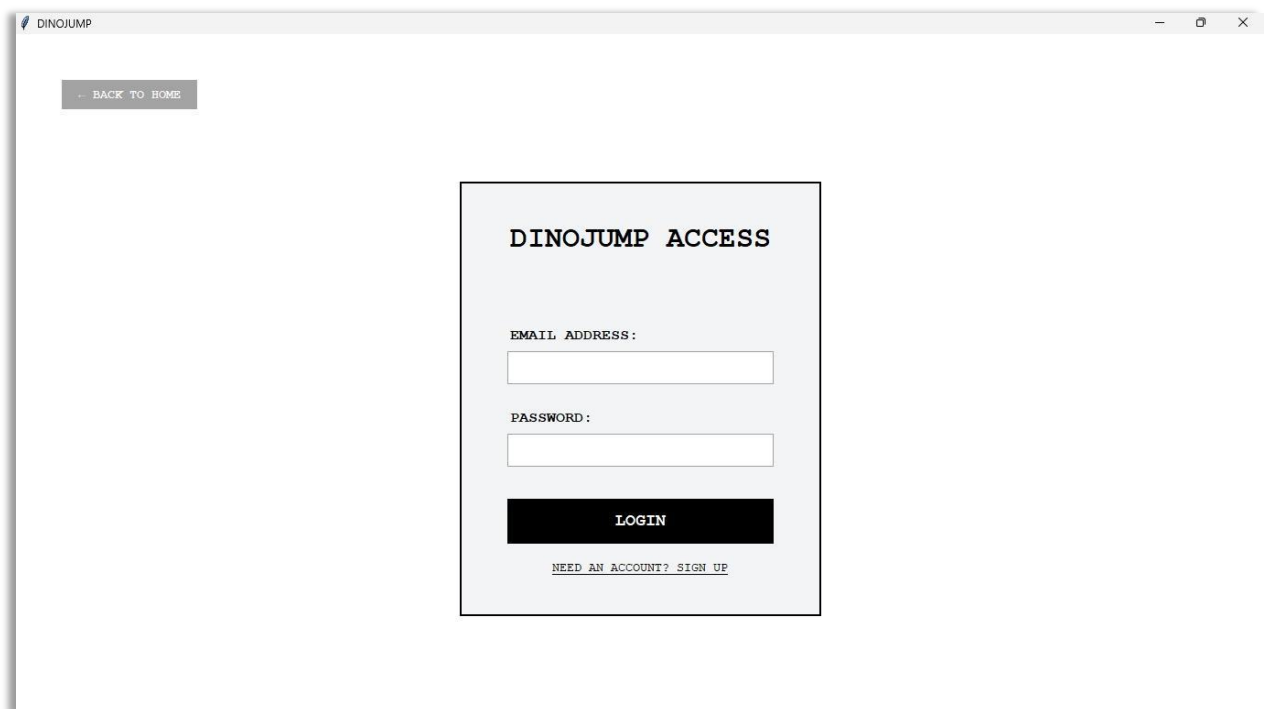


Sign in Page:



A screenshot of a web browser window titled "DINOJUMP". In the top-left corner, there is a button labeled "BACK TO HOME". The main content area features a light gray box with the title "DINOJUMP ACCESS". Inside this box, there are two input fields: "EMAIL ADDRESS:" and "PASSWORD:". Below the password field is a black button with the text "SIGN UP". At the bottom of the box, there is a link that reads "HAVE AN ACCOUNT? LOGIN".

Login Page:



A screenshot of a web browser window titled "DINOJUMP". In the top-left corner, there is a button labeled "BACK TO HOME". The main content area features a light gray box with the title "DINOJUMP ACCESS". Inside this box, there are two input fields: "EMAIL ADDRESS:" and "PASSWORD:". Below the password field is a black button with the text "LOGIN". At the bottom of the box, there is a link that reads "NEED AN ACCOUNT? SIGN UP".

Home Page:

The screenshot shows a web browser window titled "DINOJUMP". The page has a header with the "DINOJUMP" logo and a "LOGOUT" button. Below the header, a welcome message reads "WELCOME BACK, KUSHALREDDY.PUJARI!". A prominent black button with white text says "START GAME".

The main content area is divided into three columns:

- MY STATS:** A box containing the user's personal statistics: HIGH SCORE: 769, GLOBAL RANK: 16, and COUNTRY RANK: 4. At the bottom, it indicates "(COUNTRY: UK)".
- GLOBAL TOP 5:** A table listing the top 5 players globally.
- COUNTRY TOP 5:** A table listing the top 5 players in the UK.

RANK	USER	SCORE
1.	NEO_MATRIX	52350
2.	CYBER_GHOST	48720
3.	QUANTUM_LEAP	45210
4.	PHANTOM_BLADE	42380
5.	VOID_WALKER	39850

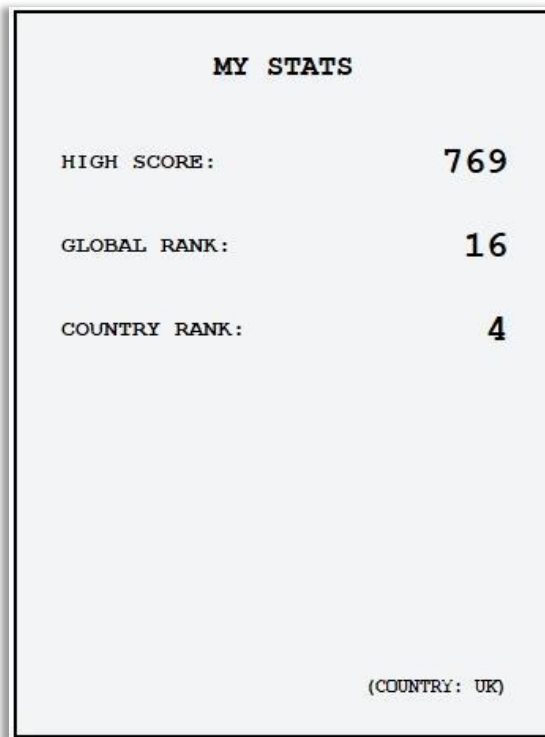
RANK	USER	SCORE
1.	CYBER_GHOST	48720
2.	ECHO_SPECTRE	35480
3.	NETRUNNER_X	26340
4.	kushalreddy.pujari	769
5.	---	---

Leaderboards:

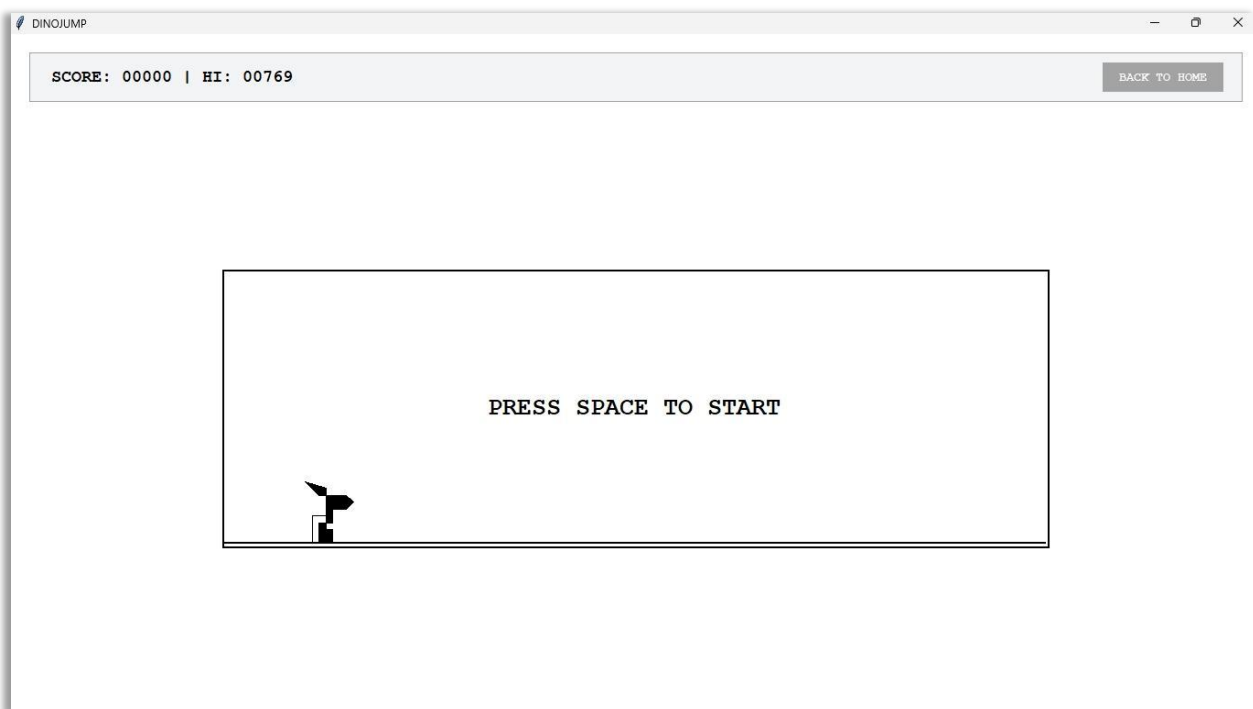
COUNTRY TOP 5		
RANK	USER	SCORE
1.	CYBER_GHOST	48720
2.	ECHO_SPECTRE	35480
3.	NETRUNNER_X	26340
4.	kushalreddy.pujari	769
5.	---	---

GLOBAL TOP 5		
RANK	USER	SCORE
1.	NEO_MATRIX	52350
2.	CYBER_GHOST	48720
3.	QUANTUM_LEAP	45210
4.	PHANTOM_BLADE	42380
5.	VOID_WALKER	39850

User Stats:



Game Page:





SQL and Tables:

```
mysql> use dinojump_db;
Database changed
mysql> show tables;
+-----+
| Tables_in_dinojump_db |
+-----+
| global_leaderboard     |
| user_achievements      |
| user_sessions          |
| users                  |
+-----+
4 rows in set (3.75 sec)
```

```
mysql> select * from global_leaderboard;
```

id	username	score	country	created_at
18	NEO_MATRIX	52350	US	2025-11-30 13:59:30
19	CYBER_GHOST	48720	UK	2025-11-30 13:59:30
20	QUANTUM_LEAP	45210	CA	2025-11-30 13:59:30
21	PHANTOM_BLADE	42380	INT	2025-11-30 13:59:30
22	VOID_WALKER	39850	JP	2025-11-30 13:59:30
23	SHADOW_STRIKE	37520	US	2025-11-30 13:59:30
24	ECHO_SPECTRE	35480	UK	2025-11-30 13:59:30
25	BINARY_STORM	33210	CA	2025-11-30 13:59:30
26	CODE_BREAKER	31240	INT	2025-11-30 13:59:30
27	SYNTAX_ERROR	29560	JP	2025-11-30 13:59:30
28	DATA_STREAM	27890	US	2025-11-30 13:59:30
29	NETRUNNER_X	26340	UK	2025-11-30 13:59:30
30	ZERO_COOL	24870	CA	2025-11-30 13:59:30
31	CRYPTO_KING	23520	INT	2025-11-30 13:59:30
32	PIXEL_PERFECT	22180	JP	2025-11-30 13:59:30
33	kushalreddy.pujari	769	UK	2025-11-30 15:32:23

16 rows in set (0.15 sec)

```
mysql> select * from user_achievements;  
Empty set (0.23 sec)
```

```
mysql> select * from user_sessions;
```

id	user_id	login_time	logout_time	ip_address	device_info
1	1	2025-11-30 14:31:55	2025-11-30 14:33:58	127.0.0.1	Desktop
2	1	2025-11-30 15:16:15	NULL	127.0.0.1	Desktop
3	1	2025-11-30 15:32:03	NULL	127.0.0.1	Desktop
4	1	2025-11-30 16:03:07	2025-11-30 16:03:35	127.0.0.1	Desktop
5	1	2025-11-30 16:33:10	2025-11-30 16:33:36	127.0.0.1	Desktop
6	1	2025-11-30 16:46:10	2025-11-30 16:46:40	127.0.0.1	Desktop
7	1	2025-11-30 16:58:35	NULL	127.0.0.1	Desktop
8	1	2025-11-30 17:12:02	NULL	127.0.0.1	Desktop

8 rows in set (0.10 sec)

```
mysql> select * from users;
```

id	email	username	display_name	date_of_birth	bio	avatar_url	password	country	high_score
last_login	total_games_played	total_play_time	achievements	privacy_settings	terms_accepted	license_accepted	created_at		
1	kushalreddy.pujari@gmail.com	kushalreddy.pujari	kushalreddy.pujari	NULL	NULL	NULL	Kushal@7hills	UK	
769	2025-11-30 17:12:02	1	0	NULL	NULL	0	0	0	2025-11-30 00:01:11

1 row in set (0.10 sec)

Conclusion

This project focuses on creating a comprehensive gaming application using the Python programming language, integrated with a MySQL database for data persistence. Titled “DinoJump,” it is an infinite runner game that allows users to register, compete for high scores, and view global leaderboards.

The project successfully demonstrates the integration of a Tkinter-based GUI with a backend database, creating a seamless user experience. It covers essential programming concepts including object-oriented design, event handling, animation, collision detection, and database CRUD operations.

The project proved to be Immensely insightful into game development principles, database management, and creating polished user interfaces with Python.

Limitations

- ❖ The game graphics are basic polygon shapes; more sophisticated sprite-based graphics could enhance visual appeal.
- ❖ The game currently lacks sound effects and background music.
- ❖ User passwords are stored in plain text in the database; encryption should be implemented for security.
- ❖ The game difficulty progression could be further refined for a smoother experience.
- ❖ The application does not include password recovery functionality.

Bibliography

❖ **“Stack Overflow.”** Stack Overflow, stackoverflow.com

❖ **“Tkinter Documentation.”**

<https://docs.python.org/3/library/tkinter.html>

❖ **“MySQL Connector/Python Developer Guide.**

<https://dev.mysql.com/doc/connector-python/en/>

❖ **“GeeksforGeeks.”** [GeeksforGeeks.com](https://www.geeksforgeeks.com)

❖ **“Python Official Documentation.**

<https://docs.python.org/3/>